

CHAPTER TWO

System Development Life Cycle & Approaches

The software process

- ❖ Software process: organizing a structured set of **activities to develop software systems**.
- ❖ Many different software processes but all involve the following activities:
 - Specification – defining what the system should do;
 - Design and implementation – defining the organization of the system and implementing the system;
 - Validation – checking that it does what the customer wants;
 - Evolution – changing the system in response to changing customer needs.

SYSTEMS DEVELOPMENT LIFE CYCLE (SDLC)

- ***Systems development life cycle (SDLC)*** - a structured step-by-step approach for developing information systems
- Typical activities include:
 - Determining budgets
 - Gathering business requirements
 - Designing models
 - Writing user documentation

SYSTEMS DEVELOPMENT LIFE CYCLE (SDLC)

SDLC PHASE	ACTIVITIES
1. Planning	•Define the system to be developed •Set the project scope •Develop the project plan
2. Analysis	•Gather business requirements
3. Design	•Design the technical architecture •Design system models
4. Development	•Build technical architecture •Build databases and programs
5. Testing	•Write test conditions •Perform testing
6. Implementation	•Write user documentation •Provide training
7. Maintenance	•Build a help desk •Support system changes

SYSTEMS DEVELOPMENT LIFE CYCLE (SDLC)



Phase 1: Planning

- ***Planning phase*** - involves determining a solid plan for developing your information system
- Three primary planning activities:
 1. Define the system to be developed
 - ***Critical success factor (CSF)*** - a factor simply critical to your organization's success

Phase 1: Planning

2. Set the project scope

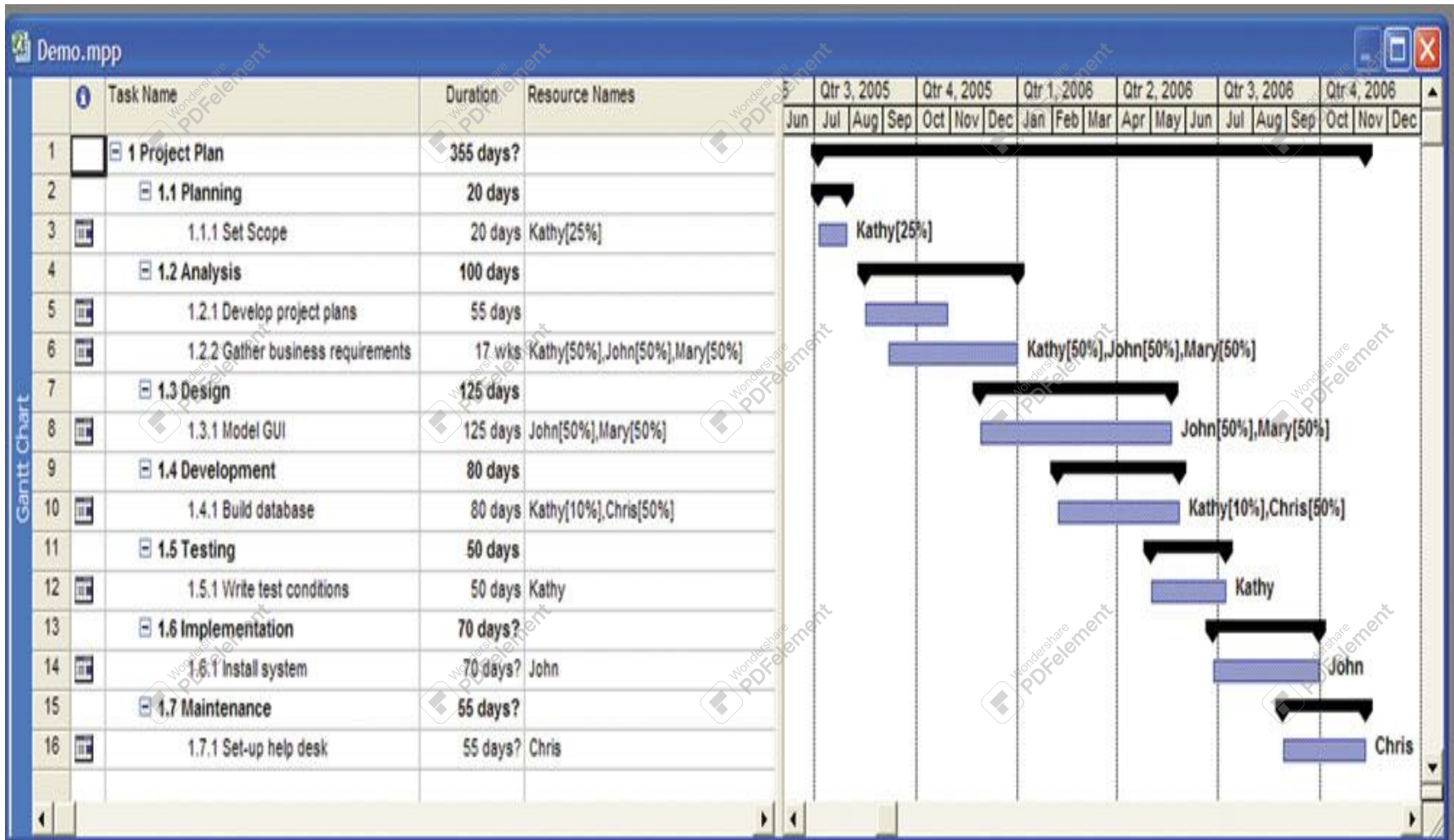
- **Project scope** - clearly defines the high-level system requirements
- **Scope creep** - occurs when the scope of the project increases
- **Feature creep** - occurs when developers add extra features that were not part of the initial requirements
- **Project scope document** - a written definition of the project scope and is usually no longer than a paragraph

Phase 1: Planning

3. Develop the project plan including tasks, resources, and timeframes

- **Project plan** - defines the what, when, and who questions of system development
- **Project manager** - an individual who is an expert in project planning and management, defines and develops the project plan and tracks the plan to ensure all key project milestones are completed on time
- **Project milestones** - represent key dates for which you need a certain group of activities performed

Phase 1: Planning



Phase 2: Analysis

- ***Analysis phase*** - involves end users and IT specialists working together to gather, understand, and document the business requirements for the proposed system

Phase 2: Analysis

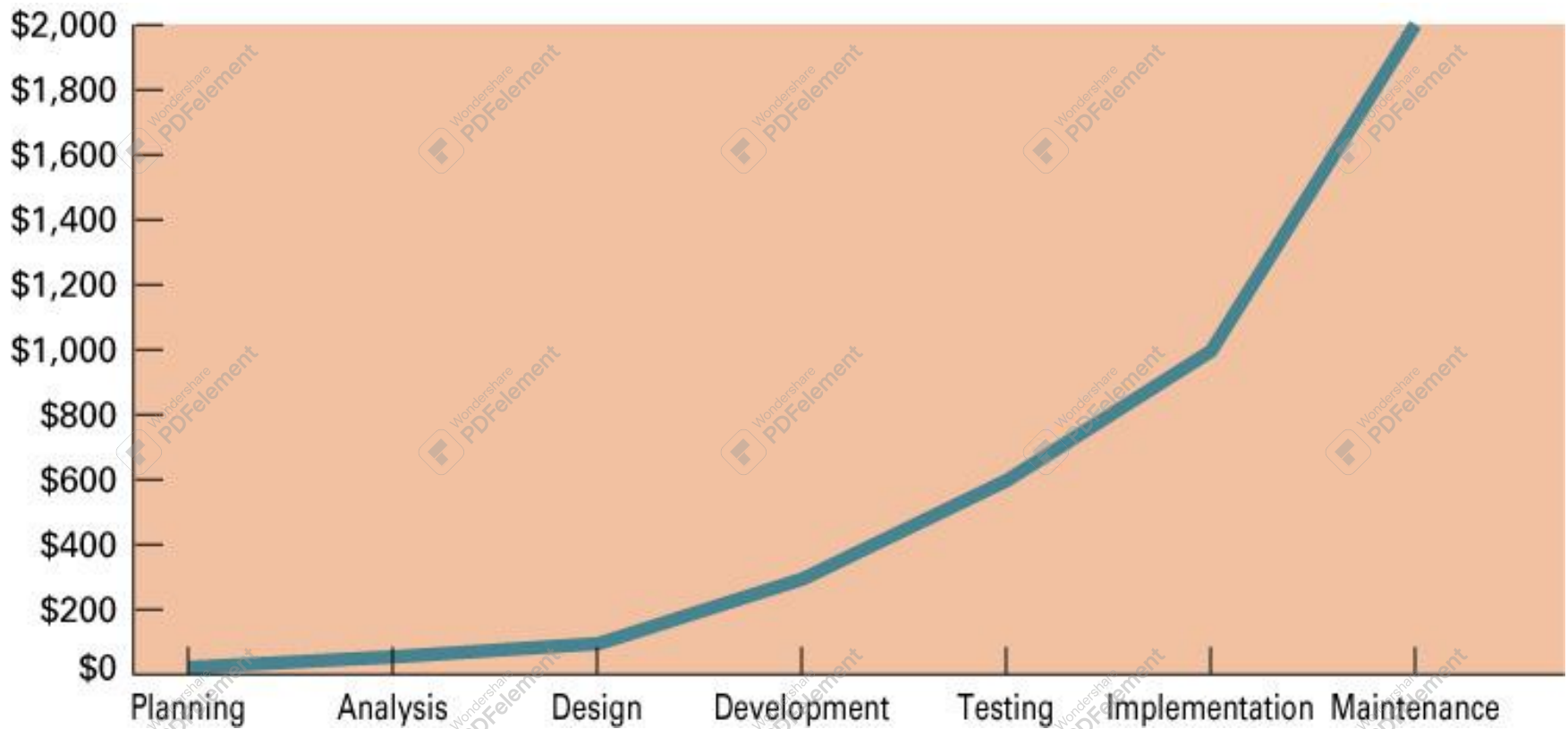
- Two primary analysis activities:
 1. Gather the business requirements
 - ***Business requirements*** - the detailed set of knowledge worker requests that the system must meet in order to be successful
 - ***Joint application development (JAD)*** - knowledge workers and IT specialists meet, sometimes for several days, to define or review the business requirements for the system

Phase 2: Analysis

2. Prioritize the requirements

- ***Requirements definition document*** – prioritizes the business requirements and places them in a formal comprehensive document

Phase 2: Analysis



Phase 3: Design

- ***Design phase*** - build a technical blueprint of how the proposed system will work
- Two primary design activities:
 1. Design the technical architecture
 - ***Technical architecture*** - defines the hardware, software, and telecommunications equipment required to run the system

Phase 3: Design

2. Design system models

- ***Modeling*** - the activity of drawing a graphical representation of a design
- ***Graphical user interface (GUI)*** - the interface to an information system
- ***GUI screen design*** - the ability to model the information system screens for an entire system

Phase 4: Development

- ***Development phase*** - take all of your detailed design documents from the design phase and transform them **into an actual system**
- Two primary development activities:
 1. Build the technical architecture
 2. Build the database and programs
 - Both of these activities are mostly performed by IT specialists

Phase 5: Testing

- **Testing phase** - verifies that the system works and meets all of the business requirements defined in the analysis phase
- Two primary testing activities:
 1. Write the test conditions
 - **Test conditions** - the detailed steps the system must perform along with the expected results of each step

Phase 5: Testing

2. Perform the testing of the system

- ***Unit testing*** – tests individual units of code
- ***System testing*** – verifies that the units of code function correctly when integrated
- ***Integration testing*** – verifies that separate systems work together
- ***User acceptance testing (UAT)*** – determines if the system satisfies the business requirements

Phase 6: Implementation

- ***Implementation phase*** - distribute the system to all of the knowledge workers and they **begin using the system** to perform their everyday jobs
- Two primary implementation activities
 1. Write detailed user documentation
 - ***User documentation*** - highlights how to use the system

Phase 6: Implementation

2. Provide training for the system users

- ***Online training*** - runs over the Internet or off a CD-ROM
- ***Workshop training*** - is held in a classroom environment and lead by an instructor

Phase 6: Implementation

- Choose the right implementation method
 - ***Parallel implementation*** – use both the old and new system simultaneously
 - ***Plunge implementation*** – discard the old system completely and use the new
 - ***Pilot implementation*** – start with small groups of people on the new system and gradually add more users
 - ***Phased implementation*** – implement the new system in phases

Phase 7: Maintenance

- ***Maintenance phase*** - monitor and support the new system to ensure it continues to meet the business goals
- Two primary maintenance activities:
 1. Build a help desk to support the system users
 - ***Help desk*** - a group of people who responds to knowledge workers' questions
 2. Provide an environment to support system changes

The Software Process Model

- ❖ A software process model is an **abstract representation** of a process
- ❖ It presents a description of a **process from some particular perspective**
- ❖ it is the mechanism of dividing software development work into distinct phases to improve design, product management, and project management.
- ❖ The models specify the stages and order of a process.

Continued...

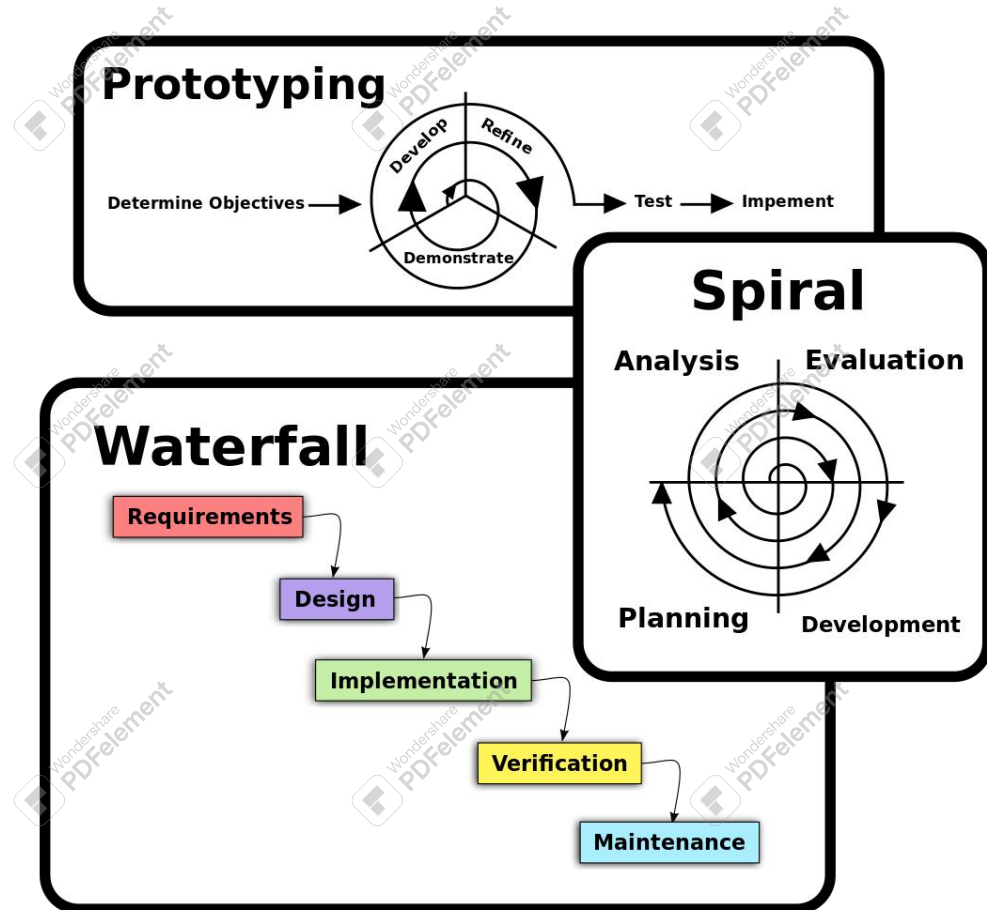
A model will define the following:

- ❖ The **tasks** to be performed
- ❖ The **input and output** of each task
- ❖ The **pre and post conditions** for each task
- ❖ The **flow and sequence** of each task

Continued...

The most popular and important **SDLC models** are as follows:

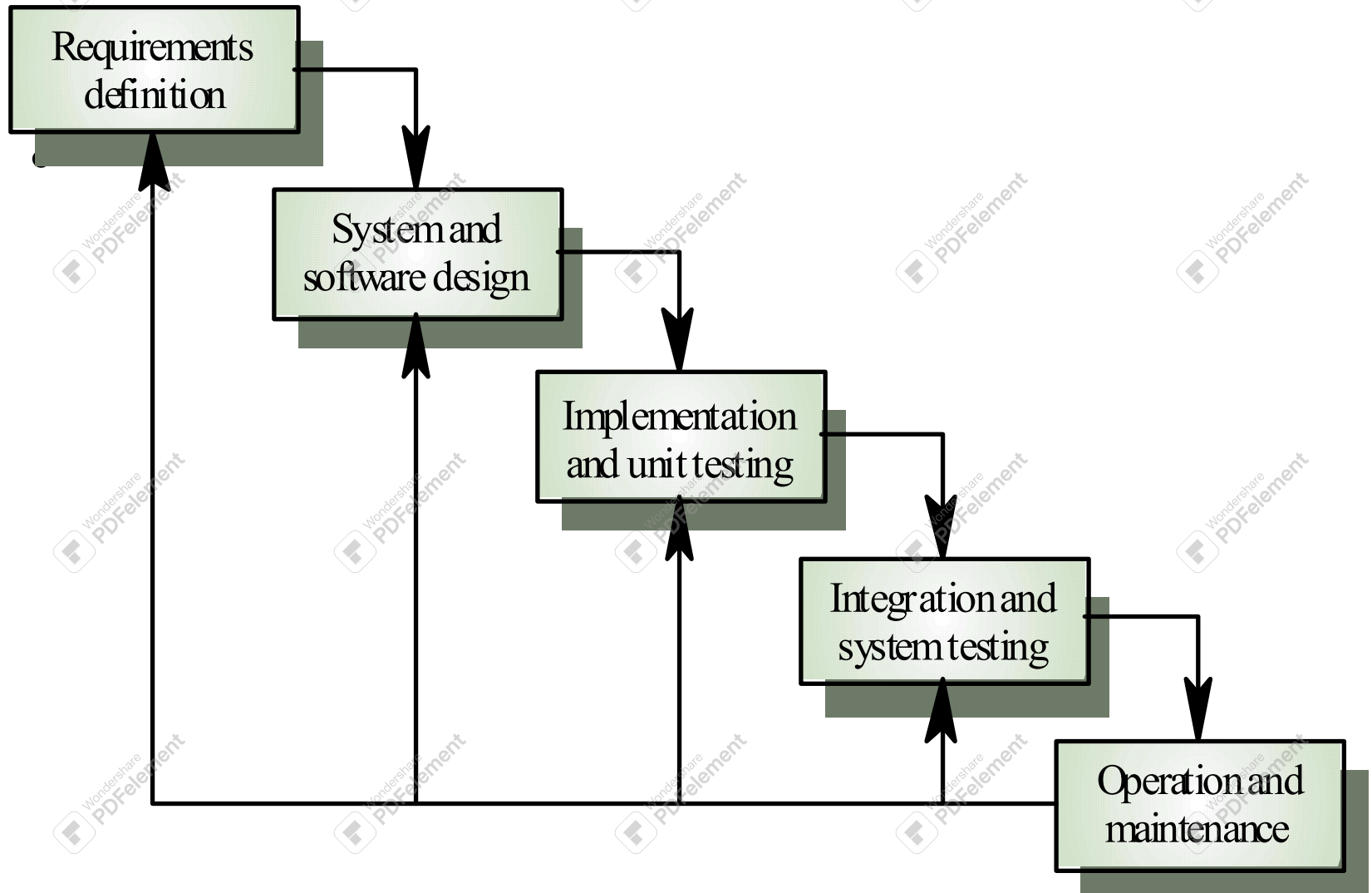
- ❖ Waterfall model
- ❖ V model
- ❖ Incremental model
- ❖ RAD model
- ❖ Agile model
- ❖ Iterative model
- ❖ Prototype model
- ❖ Spiral model



1. Waterfall Model

- ❖ The waterfall model is the first approach to software development.
- ❖ It involves moving down through the linear development stages in order: analysis, design, development, testing, deployment and maintenance.
- ❖ This model is sequentially linear meaning the next step cannot begin until the current stage is complete.

Continued...



Waterfall model phases

- ❖ Requirements analysis and definition
- ❖ System and software design
- ❖ Implementation and unit testing
- ❖ Integration and system testing
- ❖ Operation and maintenance
- ❖ The drawback of the waterfall model is the difficulty of accommodating change after the process is underway.

Waterfall model...

- ❖ There is no flexibility in this model b/c you cannot skip, backtrack or overlap the stages.
- ❖ As a result, adjusting is difficult and expensive. This lack of flexibility makes this model more costly and time consuming than other models.
- ❖ This model is not ideal for long, complex or ongoing projects where more flexibility is required.

When to use the Waterfall Model

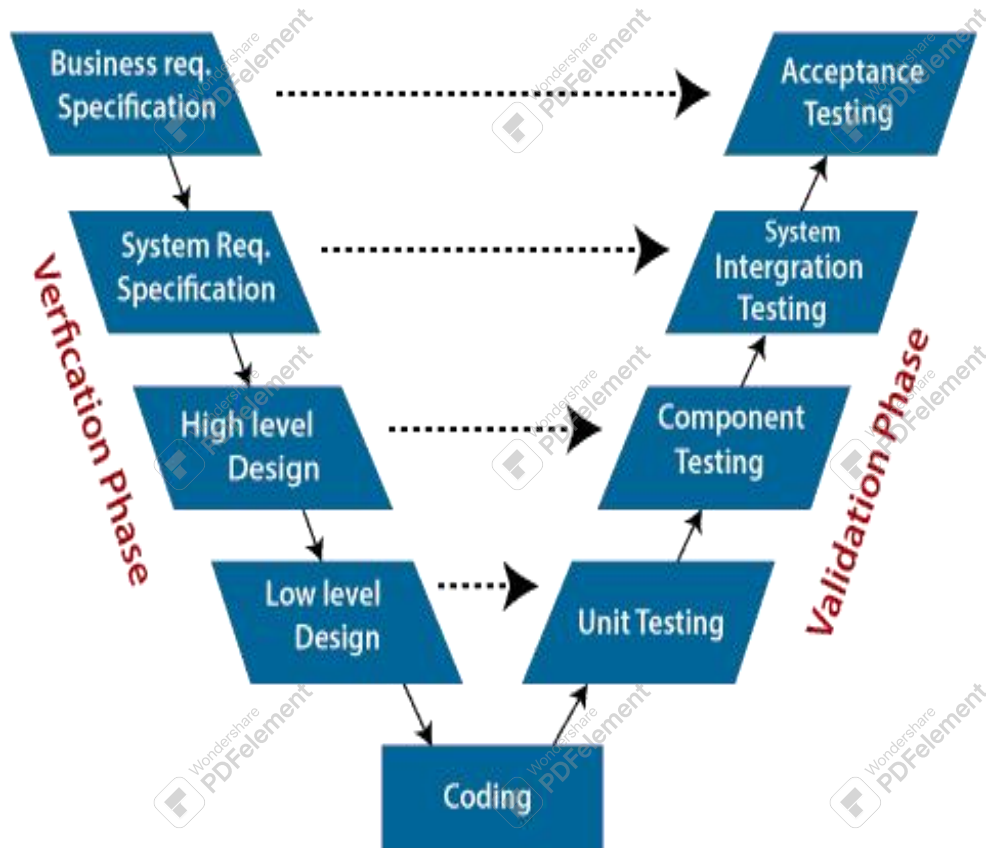
- ❖ Requirements are very **well defined, unambiguous and fixed.**
- ❖ It can be useful for small, one-time projects with few requirements and short timelines.

2. V-Shaped Model

V- Model

Developer's life Cycle

Tester's Life Cycle



- ❖ A variant of the Waterfall that emphasizes the **verification and validation** of the product.
- ❖ Testing of the product is planned in parallel with a corresponding phase of development.
- ❖ Each development stage has corresponding **testing activity** w/c allows the team to detect errors in the early stages that has a greater chance of success than waterfall model.

V-shaped...

- ❖ Like the waterfall model, a team can only **start the next stage when the current stage is complete.**
- ❖ This makes the adjustments difficult, expensive and time-intensive.
- ❖ This model is best for short projects with fixed, clearly defined, and documented requirements.
- ❖ It is not ideal for long, complex and ongoing projects.

V-Shaped Steps

- **Project and Requirements Planning** – allocate resources
- **Product Requirements and Specification Analysis** – complete specification of the software system
- **Architecture or High-Level Design** – defines how software functions fulfill the design
- **Detailed Design** – develop algorithms for each architectural component
- **Production, operation and maintenance** – provide for enhancement and corrections
- **System and acceptance testing** – check the entire software system in its environment
- **Integration and Testing** – check that modules interconnect correctly
- **Unit testing** – check that each module acts as expected
- **Coding** – transform algorithms into software

V-Shaped Strengths

- Emphasize planning for **verification and validation** of the product in early stages of product development
- Each deliverable must be testable
- Project management can track progress by milestones
- Easy to use

V-Shaped Weaknesses

- Does not easily handle concurrent events
- Does not handle iterations or phases
- Does not easily handle dynamic changes in requirements
- Does not contain risk analysis activities

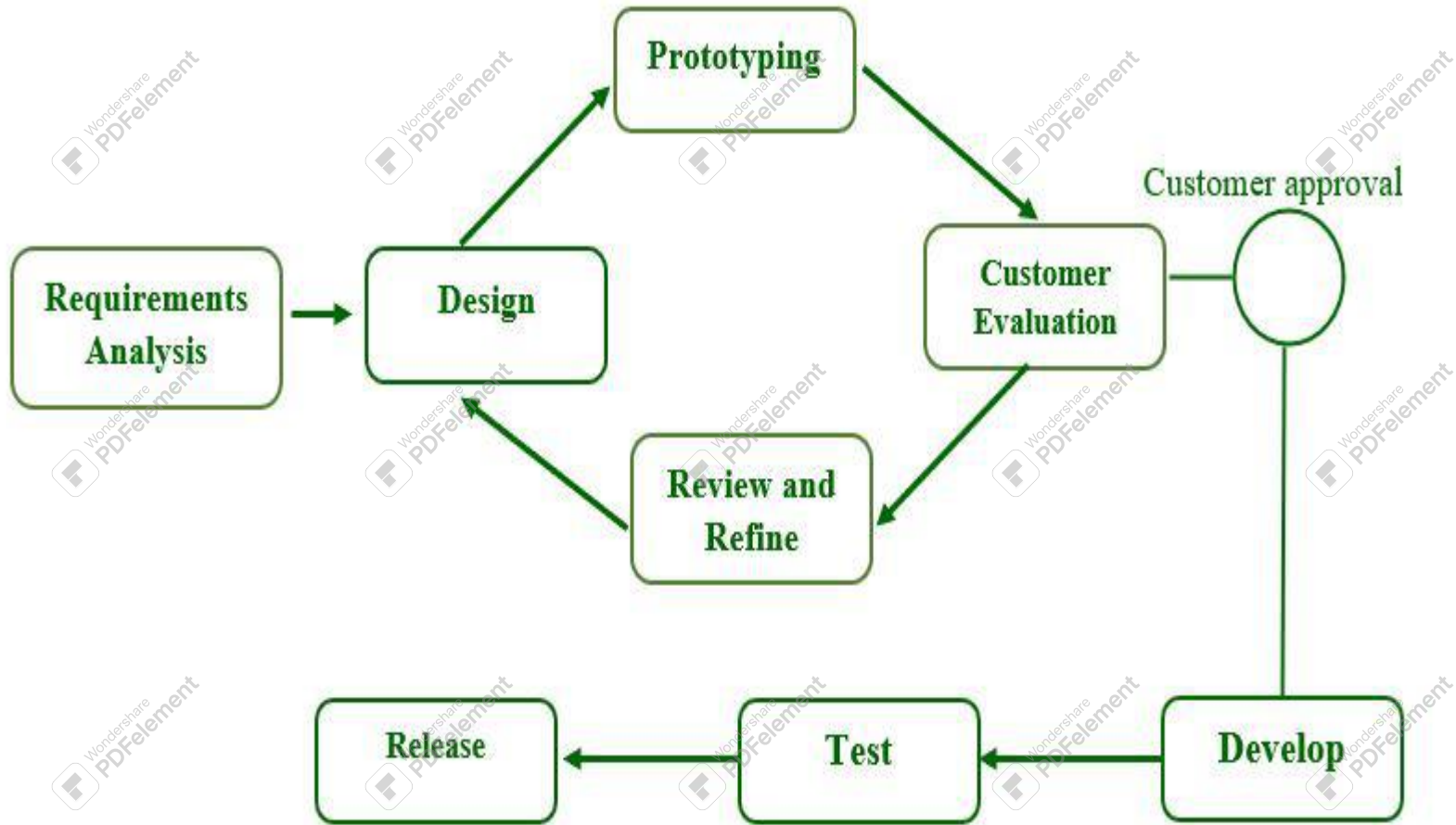
When to use the V-Shaped Model

- Excellent choice for **systems requiring high reliability** – hospital patient control applications
- **All requirements are known** up-front
- When it can be modified to **handle changing requirements beyond analysis phase**
- **Solution and technology are known**

3. Structured Evolutionary Prototyping Model

- Developers build a **prototype** (*a working small scale replica of the desired software program*) during the requirements phase
- Prototype is **evaluated by end users**
- Users give **corrective feedback**
- Developers further **refine the prototype**
- When the **user is satisfied**, the prototype code is brought up to the standards needed for a final product.

Prototyping Model...



Structured Evolutionary Prototyping Steps

- A preliminary project plan is developed
- A partial high-level paper model is created
- The model is source for a partial requirements specification
- A prototype is built with basic and critical attributes
- The designer builds
 - the database
 - user interface
 - algorithmic functions
- The designer demonstrates the prototype, the user evaluates for problems and suggests improvements.
- This loop continues until the user is satisfied

Structured Evolutionary Prototyping Strengths

- Customers can “see” the system requirements as they are being gathered
- Developers learn from customers
- A more accurate end product
- Allows for flexible design and development
- Interaction with the prototype stimulates awareness of additional needed functionality

Structured Evolutionary Prototyping Weaknesses

- Tendency to abandon structured program development for “code-and-fix” development
- Process may continue forever (scope creep)
- When the final prototype is in development, no more requirements or alterations can be made to the requirements.

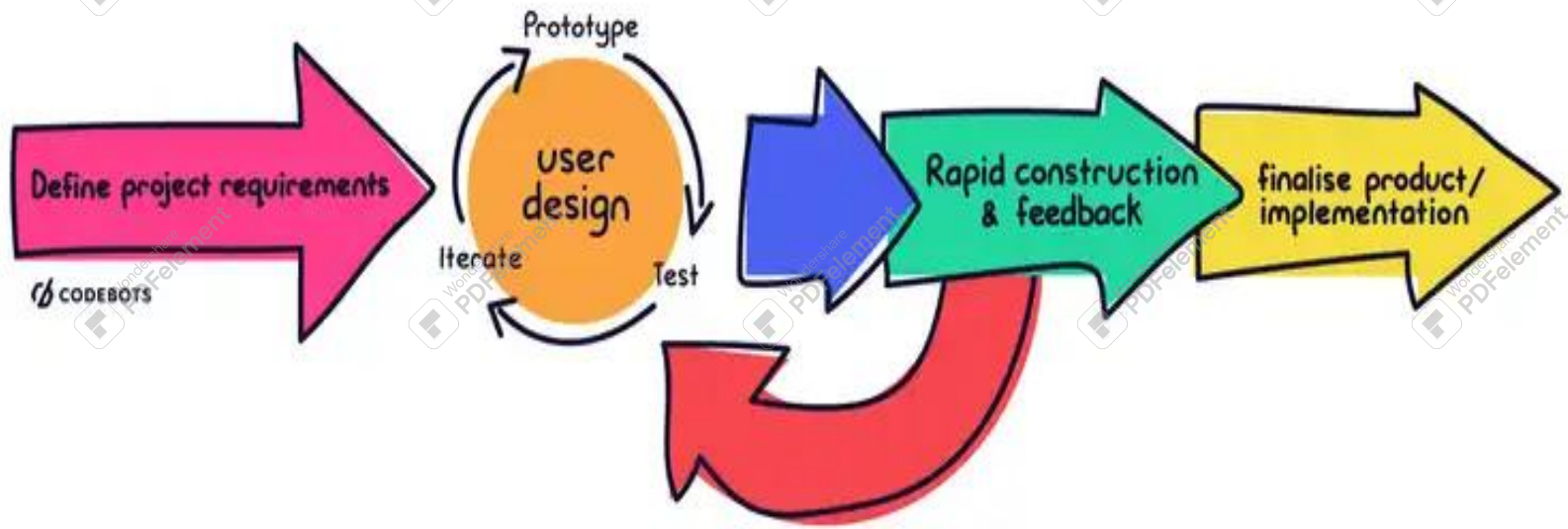
When to use Structured Evolutionary Prototyping

- Short-lived demonstrations
- New, original development
- With the analysis and design portions of object-oriented development.

4. Rapid Application Development Model (RAD)

- In this model there is a delay in producing system design **until user requirements are clear.**
- 1. **Requirements planning phase** (a workshop utilizing structured discussion of business problems)
- 2. **User description phase** – automated tools capture information from users
- 3. **Construction phase** – productivity tools, such as code generators, screen generators, etc. inside a time-box. (“Do until done”)
- 4. **Cutover phase** -- installation of the system, user acceptance testing and user training

RAD Model...



RAD Strengths

- **Reduced cycle time** and improved productivity with fewer people means lower costs
- **Customer involved throughout** the complete cycle minimizes risk of not achieving customer satisfaction and business needs
- **Uses modeling concepts** to capture information about business, data, and processes.

RAD Weaknesses

- Accelerated development process **must give quick responses** to the user
- Hard to use with **legacy systems**
- Developers and customers must be **committed to rapid-fire activities** in an abbreviated time frame.

When to use RAD

- Reasonably well-known requirements
- User involved throughout the life cycle
- Project can be time-boxed
- High performance not required
- Low technical risks

5. Incremental Development

- It is a process of software development where **requirements divided into multiple standalone modules** of the software development cycle.
- Each module goes through the requirements, design, implementation and testing phases.
- Every subsequent release of the module adds function to the previous release.
- The process continues until the complete system achieved.

Incremental Development

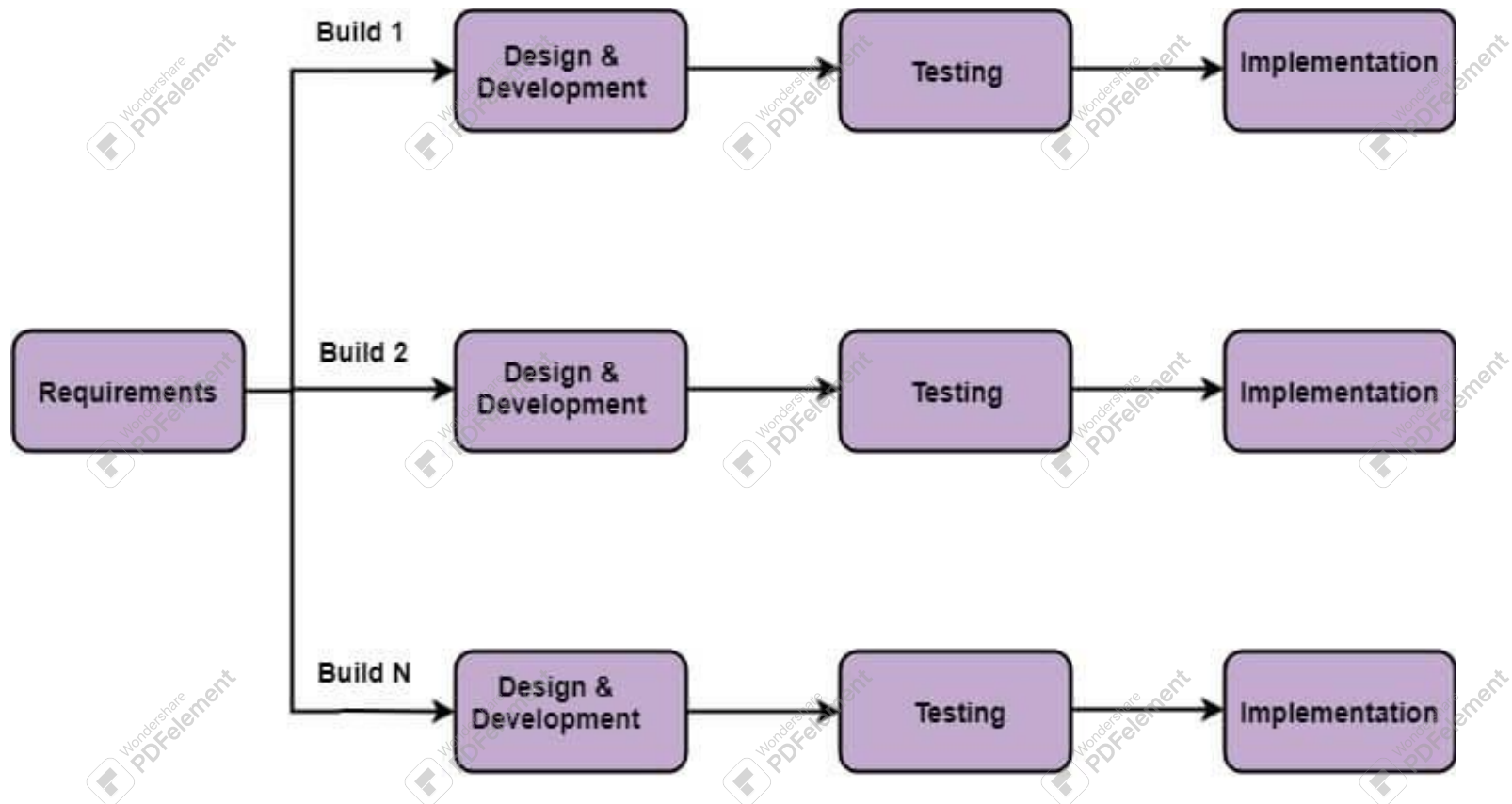


Fig: Incremental Model

Incremental Development –Advantages

- **Customer value** can be delivered with each increment so system functionality is available earlier.
- **Early increments** act as a prototype to help elicit requirements for later increments.
- **Lower risk of overall project failure.**
- **The highest priority system services** tend to receive the most testing.

Incremental Model Strengths

- Each release delivers an **operational product**
- Customer can **respond to each build**
- Uses “divide and conquer” **breakdown of tasks**
- Lowers **initial delivery cost**
- Initial **product delivery is faster**
- Customers get **important functionality early**
- Risk of **changing requirements is reduced**

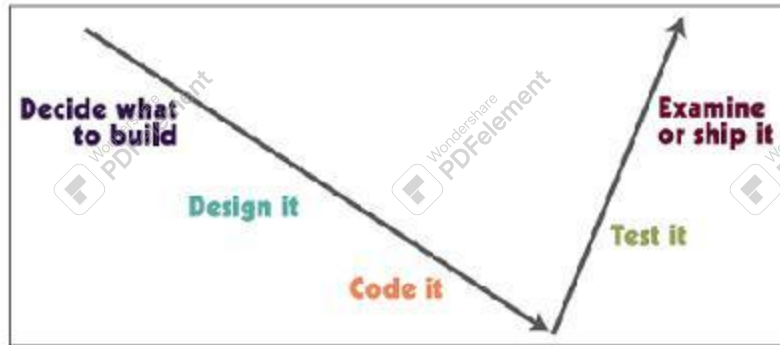
Incremental Development – Problems

- Lack of process visibility.
- Systems are often poorly structured.

When to use the Incremental Model

- Risk, funding, schedule, program complexity, or need for **early realization of benefits**.
- Most of the requirements are known up-front but are expected to **evolve over time**
- A need to **get basic functionality to the market early**
- On projects which have **lengthy development schedules**
- On a project with **new technology**

Incremental Development



- The first increment delivers one slice of functionality through the whole system.
- The second increment delivers another slice of functionality through the whole system.
- The third increment delivers the rest of the system

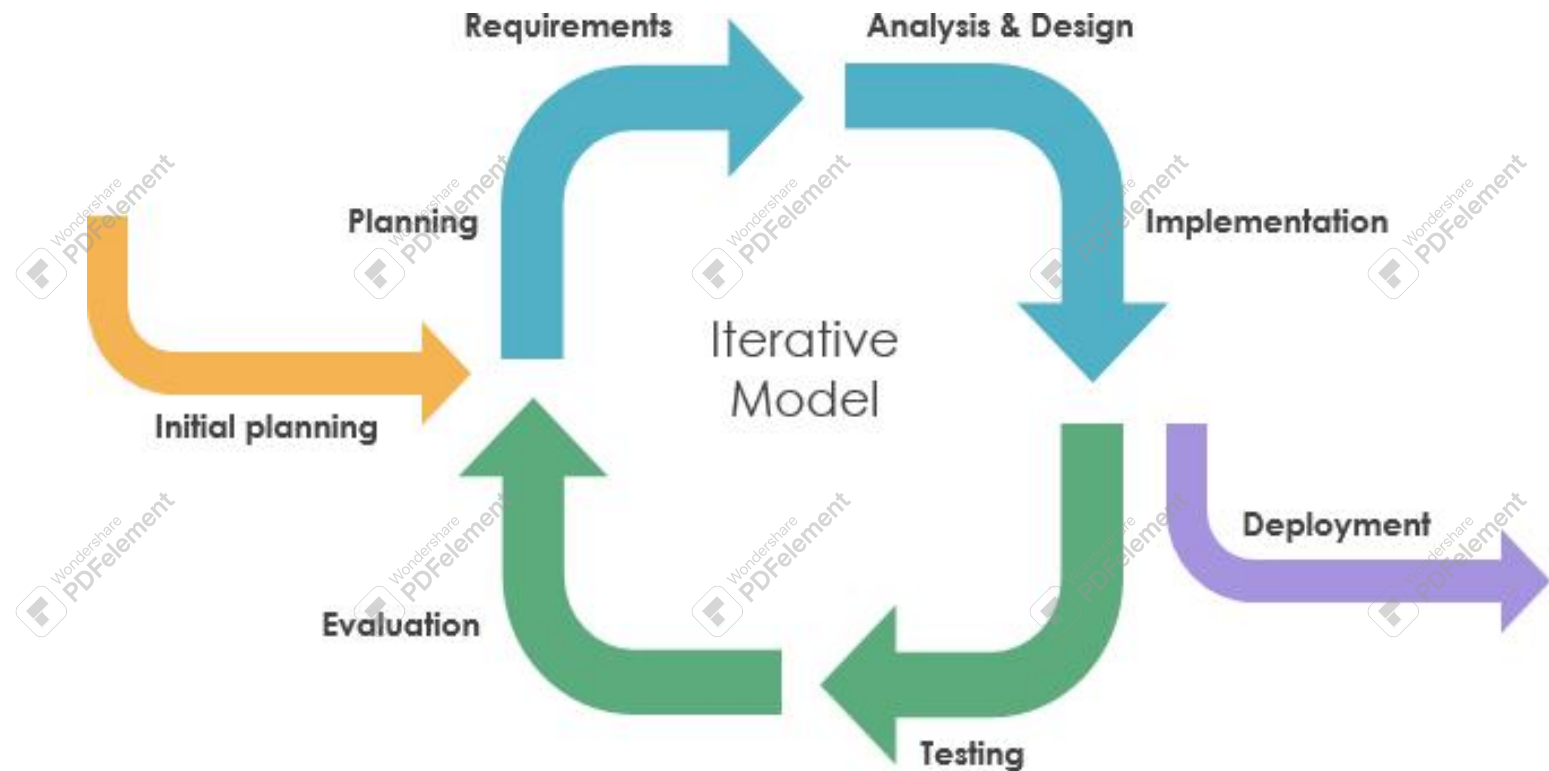
	Feature 1	Feature 2	Feature 3
UI layer			
App layer			
Middleware			
Database			

	Feature 1	Feature 2	Feature 3
UI layer			
App layer			
Middleware			
Database			

	Feature 1	Feature 2	Feature 3
UI layer			
App layer			
Middleware			
Database			

6. Iterative Development

- This model iterates requirements, design, build and test phases again and again for each requirement and builds up a system iteratively till the system is completely build.



6. Iterative Development

- The first iteration delivers enough of feature 1 to evaluate what is correct and what needs revision.
- After the second iteration, some revised parts still need improvement.
- The third iteration produces the final and stable feature

	Feature 1	Feature 2
UI layer	?	
App layer	?	
Middleware	✓	
Database	?	

	Feature 1	Feature 2
UI layer	??	
App layer	??	
Middleware	✓	
Database	✓	

	Feature 1	Feature 2
UI layer	✓	
App layer	✓	
Middleware	✓	
Database	✓	

Problems with incremental development (from a waterfall eye...)

- **Management problems**

- Progress can be hard to judge and problems hard to find because there is no documentation to demonstrate what has been done.

- **Contractual problems**

- The normal contract may include a specification; without a specification, different forms of contract have to be used.

- **Validation problems**

- Without a specification, what is the system being tested against?

- **Maintenance problems**

- Continual change tends to corrupt software structure making it more expensive to change and evolve to meet new requirements.

Incremental vs. iterative model

- Incremental means adding, iterative means reworking
- In incremental development the various parts of the system are developed at different times or rates and integrated as they are completed.
- Iterative development is a rework scheduling strategy in which time is set aside to revise and improve parts of the system.

Increment	Iterate
fundamentally means “ <i>add onto</i> ”.	fundamentally means “ <i>change</i> ”.
repeating the process on a <i>new</i> section of work.	repeating the process on the <i>same</i> section of work

Getaneh M.

7. Spiral Development

- It combines features of **waterfall and prototyping** models.
- The spiral model focuses on **risk assessment**.
- **Process is conceived as a spiral** rather than as a sequence of activities with backtracking.
- **Each loop in the spiral represents a phase** in the process.
- **No fixed phases such as specification or design** - loops in the spiral are chosen depending on what is required.

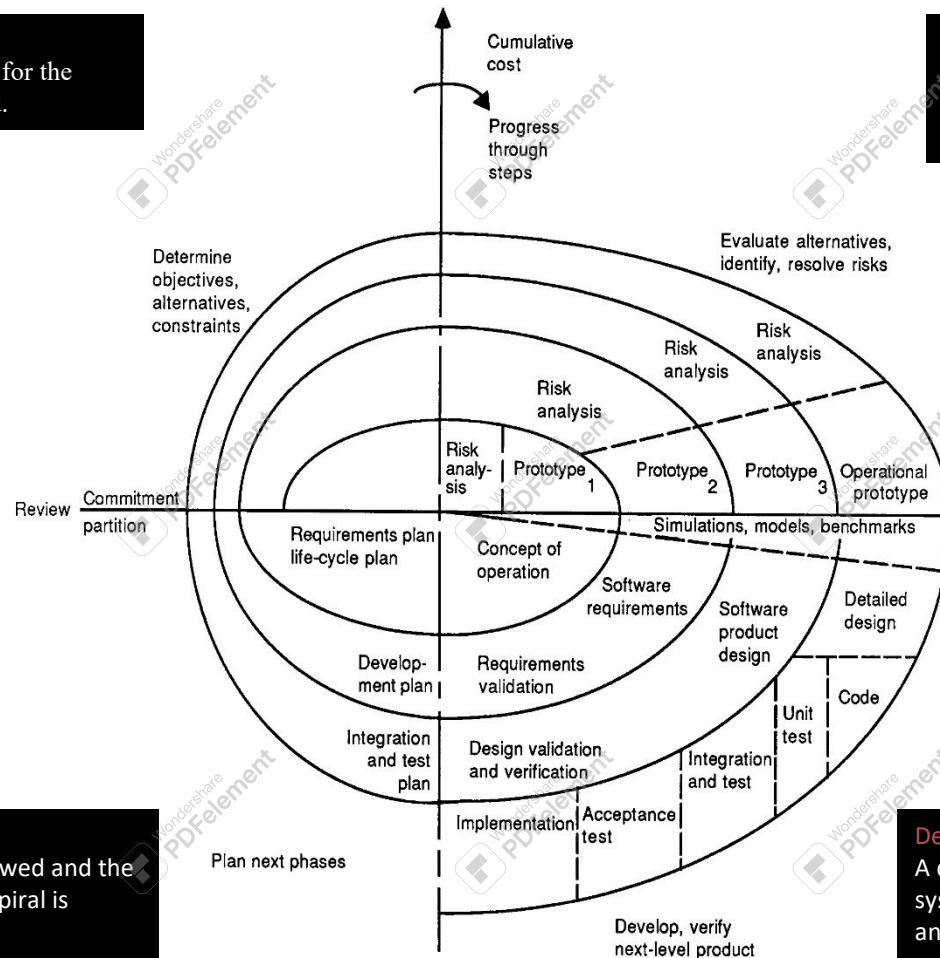
Spiral model (Boehm 87)

Objective setting

Specific objectives for the phase are identified.

Risk assessment and reduction

Risks are assessed and activities put in place to reduce the key risks.



Planning

The project is reviewed and the next phase of the spiral is planned.

Development and validation

A development model for the system is chosen which can be any of the generic models.

Spiral model sectors

- **Objective setting**
 - Specific objectives for the phase are identified.
- **Risk assessment and reduction**
 - Risks are assessed and activities put in place to reduce the key risks.
- **Development and validation**
 - A development model for the system is chosen which can be any of the generic models.
- **Planning**
 - The project is reviewed and the next phase of the spiral is planned.

Spiral Model Strengths

- Provides early indication of risks, without much cost
- Users see the system early because of rapid prototyping tools
- Critical high-risk functions are developed first
- The design does not have to be perfect
- Users can be closely tied to all lifecycle steps
- Early and frequent feedback from users

Spiral Model Weaknesses

- Time spent for evaluating risks too large for small or low-risk projects
- Time spent planning, resetting objectives, doing risk analysis and prototyping may be excessive
- The model is complex
- Risk assessment expertise is required
- May be hard to define objective, verifiable indicators that indicate readiness to proceed through the next iteration

When to use Spiral Model

- When creation of a prototype is appropriate
- When costs and risk evaluation is important
- For medium to high-risk projects
- Users are unsure of their needs
- Requirements are complex
- New product line
- Significant changes are expected (research and exploration)

End

Question???